CS 352 – Internet Technology

Project 3: Reliable UDP (RUDP)

World Cup Live Scoreboard

Total Points: 100 **Mode:** Individual or Group of 2

1 Objective

In this project, you will design and implement a simplified Reliable UDP (RUDP) protocol on top of UDP sockets. UDP's own header only has a source port, destination port, length and checksum, so you will put **your own TCP-style header inside the UDP data**. Your implementation will demonstrate how TCP achieves reliable, ordered delivery through:

- A **three-way handshake** with random initial sequence numbers (connection setup)
- **Stop-and-wait reliability** (byte sequence numbers, ACKs, timeouts, and retries)
- A **FIN / FIN-ACK teardown** (connection close)

On July 19, 2026, Spain beat Argentina 1–0 after extra time in the World Cup final at MetLife Stadium in New Jersey. Your client is the stadium: it streams that match, one event per packet, to a broadcast server that keeps the live scoreboard. The network between them drops and delays packets. If your protocol delivers Ferran Torres's 106th-minute goal twice, the broadcast shows Spain winning 2–0. If it loses the packet, the goal never happened. Your job is to make the scoreboard exactly right every time.

The server intentionally drops and delays its replies, so your client will experience timeouts and must retransmit, mimicking real network loss and latency.

2 Assigned UDP Port

Each student or team is assigned a unique UDP port on Canvas. Use this same port in both client and server.

```
ASSIGNED_PORT = 30077   # <-- REPLACE with your assigned UDP port
SERVER = ("127.0.0.1", ASSIGNED_PORT)
```

Use only your assigned port as using another team's port may cause conflicts during grading.

3 Provided Files

You are provided exactly three files:

File	What it is
`rudp_client.py`	The stadium: reads the match feed and sends it
`rudp_server.py`	The broadcast server: keeps the scoreboard
`match_feed.txt`	24 events from the final, one per line

The skeletons include the header format and helper functions you must use and must not change: `pack_msg()`, `unpack_msg()`, `seq_add()`, `log()`, `send_recv_with_retry()` (client), and on the server `send_unreliable()` and the `Scoreboard` class. You must complete all TODO sections to implement:

- Three-way handshake with random initial sequence numbers
- Reliable stop-and-wait data transfer with byte sequence numbers
- Connection teardown (FIN / FIN-ACK)

4 Protocol Specification

Every RUDP packet is an ordinary UDP packet whose data starts with this 11-byte header, followed by the payload (if any). Format string: `'!I I B H'`.

Field	Size	Meaning
seq	4 bytes	Sequence number of the first byte of this packet's payload
ack	4 bytes	Next byte the sender expects to receive (valid when the ACK flag is set)
flags	1 byte	SYN, ACK, FIN bits (below)
len	2 bytes	Payload length in bytes

Flag	Bit	Purpose
SYN	`0x01`	Client starts a connection; carries its initial sequence number
ACK	`0x02`	The ack field is valid
FIN	`0x04`	Client requests close

Packets combine flags: a SYN-ACK is `SYN | ACK`, a FIN-ACK is `FIN | ACK`.

Sequence numbers count bytes, not packets, exactly like TCP. If a DATA packet has seq = 1000 and carries 35 bytes, the next DATA has seq = 1035 and the ACK for it has ack = 1035. A SYN and a FIN each use up one sequence number, which is why the replies say $x + 1$, $y + 1$ and FIN's seq + 1. All arithmetic wraps around at 2^{32}, so always use the provided `seq_add()`.

Initial sequence numbers (ISNs) are random. The client picks `x = random.randint(0, 2**32 - 1)`; the server picks its own y the same way. Never start at 0.

Wireshark cannot decode this header, because to Wireshark it is just UDP data. That is why every packet you send or receive must be printed with the provided `log()`:

```
[CLIENT] SEND seq=3258487412 ack=0 flags=SYN len=0
[CLIENT] RECV seq=3169327252 ack=3258487413 flags=SYN|ACK len=0
```

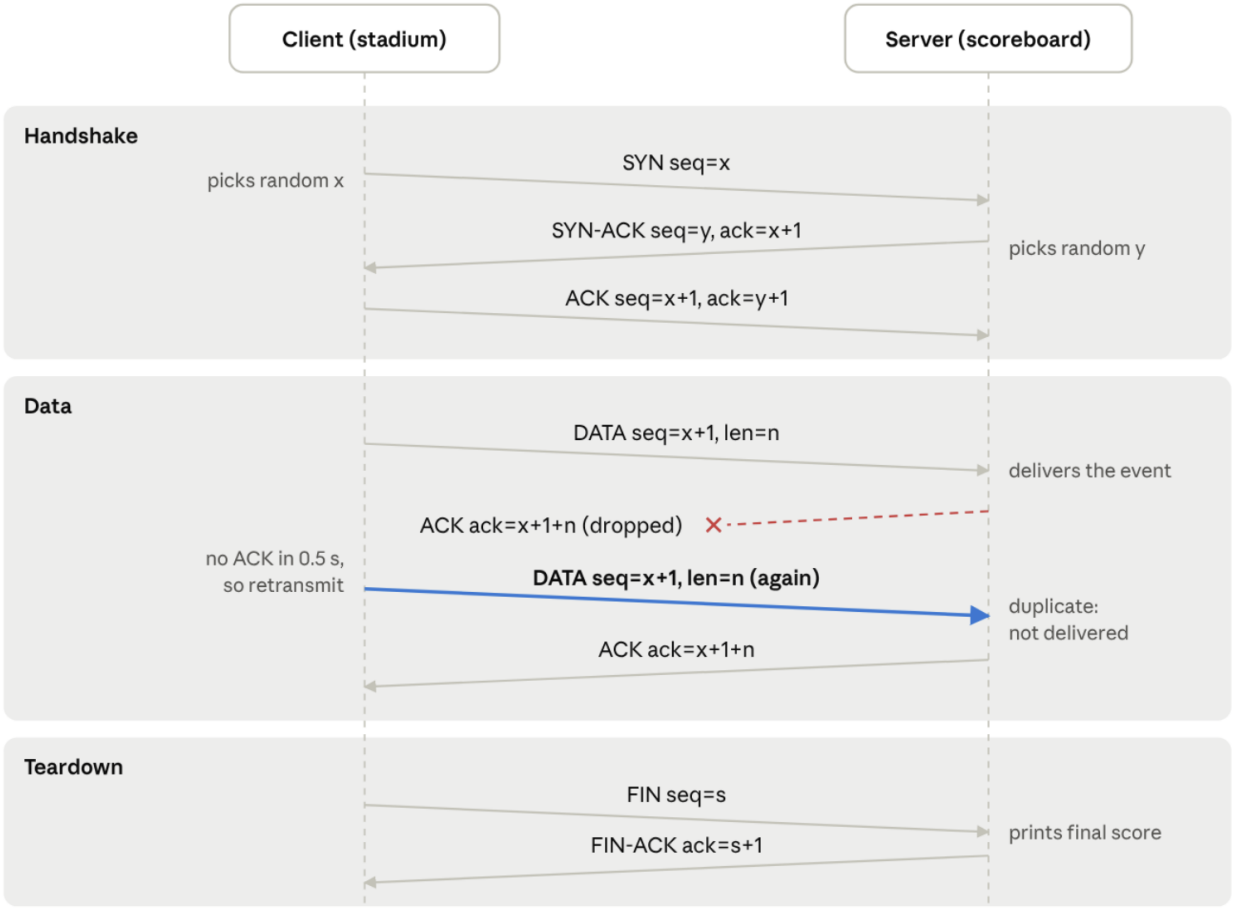


Figure 1: One full connection. x and y are the two random ISNs, n is the event's length in bytes, and s is the client's sequence number after the last event.

5 Implementation Tasks

A. Three-Way Handshake

Implement:

1. Client → Server: **SYN** (seq = x, a random ISN)
2. Server → Client: **SYN-ACK** (seq = y, its own random ISN; ack = $x + 1$)
3. Client → Server: **ACK** (seq = $x + 1$, ack = $y + 1$)

The client must accept a SYN-ACK only if its ack equals $x + 1$, and the server must complete the handshake only if the ACK's ack equals $y + 1$. If no valid SYN-ACK arrives within the timeout, the client retransmits the SYN (at most 5 tries). If the server sees a duplicate SYN, it resends the **same** SYN-ACK (same y). If the client's final ACK is lost, its first DATA packet (which also carries the ACK flag and ack = $y + 1$) completes the handshake.

After successful setup, print:

```
[CLIENT] Connection established
[SERVER] Connection established
```

B. Reliable Data Transfer (Stop-and-Wait)

Send each line of `match_feed.txt` as one DATA packet, one at a time. The first DATA has seq $= x + 1$; every DATA carries ack $= y + 1$ and the ACK flag.

- **Client:** after sending DATA(seq $= s$, len $= n$), wait for an ACK with ack $= s + n$. If none arrives within the timeout (RTO $= 0.5$ s), retransmit. Ignore ACKs with any other ack number (they are late replies to earlier packets). After 5 tries, give up with an error message.
- **Server:** keep `expect_seq`, the next byte you expect. If a DATA has seq = `expect_seq`, deliver it to the scoreboard (`board.apply()`) and add its length to `expect_seq`. Otherwise it is a duplicate: **do not deliver it**. Either way, reply with ack = `expect_seq`.
- **Server:** send every reply with `send_unreliable()`. It drops 10% of replies and holds back another 20% for 100–1000 ms ("VAR review"), so your client will time out and retransmit, and your server will receive duplicates. Do not edit it.

This is where the scoreboard matters: a server that delivers duplicates shows the same event twice, and a duplicated GOAL gives the wrong score.

C. Connection Teardown

Implement clean closing:

1. Client → Server: **FIN** (seq = the client's current seq)
2. Server → Client: **FIN-ACK** (ack = FIN's seq + 1)

The server then calls `board.save()`, which prints the final score and writes the delivered events to `received_feed.txt`. If a FIN-ACK is lost, the client retransmits the FIN, so the server must remember its last FIN-ACK and resend it. After completion, print:

```
[CLIENT] Connection closed
[SERVER] Connection closed
```

6 Running and Testing

Start the server first, then the client, in two terminals:

```
python3 rudp_server.py          # optional: python3 rudp_server.py 7  (a seed replays
    the same drops and delays)
python3 rudp_client.py match_feed.txt
```

Your run is correct when the server prints `FINAL SCORE: ESP 1  ARG 0` and `received_feed.txt` is identical to `match_feed.txt` (`diff match_feed.txt received_feed.txt` prints nothing). Run it several times: the drops and delays are random, and a bug may only show up on some runs.

7 Wireshark Capture Requirements

Capture actual packets using Wireshark on the loopback interface (`lo0` on macOS, `lo` on Linux). Apply the filter `udp.port == <your_port>` and save:

- `project3_handshake.pcap` – SYN, SYN-ACK, ACK
- `project3_data.pcap` – DATA and ACK packets, **showing at least one retransmission**
- `project3_teardown.pcap` – FIN and FIN-ACK

Wireshark will show these as plain UDP. Your header is the first 11 bytes of the UDP data, so a retransmission appears as two packets with identical bytes. Also save the terminal output of both programs for the same run as `client_log.txt` and `server_log.txt`.

8 Report Requirements

Your `report.pdf` must include:

- **Implementation Summary:** explain the handshake (with your actual x and y from one run), data transfer, and teardown logic.
- **Capture Analysis:** for one handshake packet, show the 11 header bytes in hex from Wireshark and decode each field by hand. Describe how retransmissions appear in `project3_data.pcap`.
- **Discussion:** why a dropped or delayed reply triggers a retransmission, why the server must not deliver a duplicate, and why ISNs are random rather than 0.

9 Submission

Submit a ZIP file named `<netid>_project3.zip` containing:

- `rudp_client.py`, `rudp_server.py`
- `project3_handshake.pcap`, `project3_data.pcap`, `project3_teardown.pcap`
- `client_log.txt`, `server_log.txt`
- `report.pdf`

If working in a group of 2, submit only one ZIP file with both names and NetIDs clearly mentioned in the report.

10 Grading

We will run your client against our server and your server against our client, with several different drop/delay seeds. A run passes when the final score is correct and the delivered feed matches `match_feed.txt` exactly.

Part	**Points**	**How it is checked**
A. Handshake	25	Random ISNs; ack $= x+1$ and $y+1$; SYN retransmitted on timeout
B. Data transfer	35	Exact feed and correct score on every seed; no duplicate delivered; 5-try limit
C. Teardown	10	FIN / FIN-ACK with ack = seq $+1$; duplicate FIN answered
Captures and logs	15	Three pcaps as listed; logs in the required format
Report	15	Sections in Part 8, including the hand-decoded header